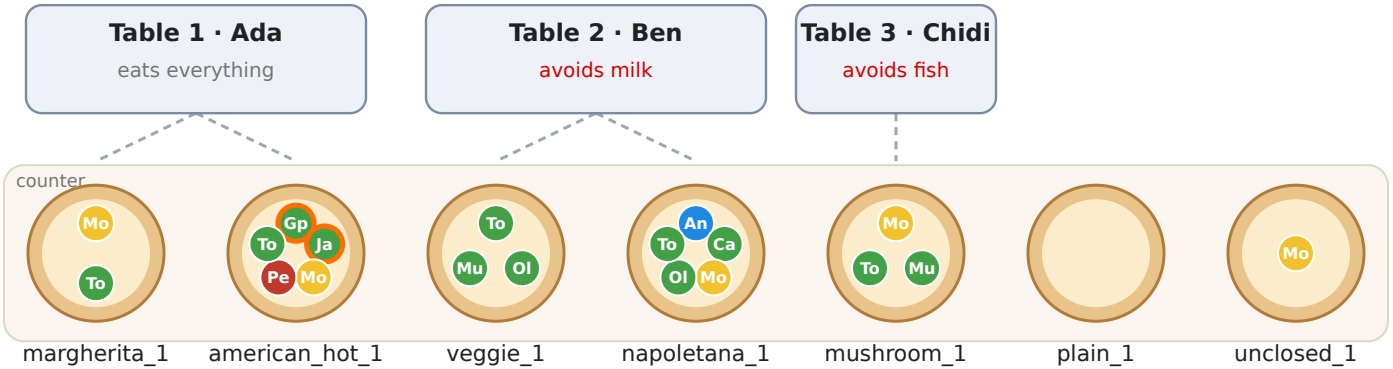
Asking, Describing, Constraining, Inferring

Worksheet 3 · querying a knowledge base with the Entity Query Language

> **Now:** in pairs, open `03-pizza_queries.ipynb` (link on Stud.IP) and run its **setup cell**. It installs EQL while we talk.

Last week: represent. Today: use.

- Worksheet 2: the pizza world in first-order logic, in OWL, and as Python classes.
- Today a service robot in a pizzeria *uses* that knowledge. Everything it does starts with one kind of expression: a **description**.



A query is a formula with free variables

$$\text{Pizza}(x) \land \exists t\,(\text{hasTopping}(x, t) \land \text{CheeseTopping}(t))$$

- The picture of the pizzeria is an **interpretation**: a domain of objects and the extension of every predicate.
- The **answers** are the values of the free variable *x* that make the formula true in it.
- That is *checking* one model, not proving that something holds in *every* model. The difference will matter: does the robot know everything?

EQL: the same formula, over Python objects

```
cheesy = a(Pizza)(toppings=a(CheeseTopping)()).from_(pizzas)
```

EQL	**first-order logic**
`a(Pizza)`	a variable x with Pizza(x)
`toppings=a(CheeseTopping)()`	$\exists t$ (hasTopping(x, t) $\wedge$ CheeseTopping(t)): *some* element of the list
`.from_(pizzas)`	the objects x ranges over
`cheesy.tolist()`	all values of x that satisfy the formula

Read it aloud: *“a pizza with a cheese topping, from the pizzas”*. Writing a query computes nothing; `.tolist()` evaluates it.

Beyond patterns: naming variables

```
pizza   = a(Pizza).from_(pizzas)
topping = flat_variable(pizza.toppings)
pizza.where(for_all(topping, HasType(topping, VegetableTopping)))
```

$$\text{Pizza}(x) \land \forall t\,(\text{hasTopping}(x, t) \rightarrow \text{VegetableTopping}(t))$$

- A pattern says *some*. For *every* and *no*, name the topping and use `for_all`, `not_(exists(…))`.
- Every `a(…)` is a variable of its own: two `a(Pizza)` are two pizzas.
- `set_of(x, y)` returns several free variables together: the answers become assignments.

One description, five uses

Act	Story	The description is …	Logic
1 Asking	“Which pizzas have cheese?”	a question	∃, ∀, ¬ in one interpretation
2 Describing	“Bring me *the* pizza with olives.”	a reference	definite descriptions
3 Constraining	“Ben must never get milk.”	a constraint	¬∀ ≡ ∃¬
4 Inferring	allergen warnings, menu labels	the body of a rule	rules, exceptions
5 Generating	Dana orders a pizza that does not exist	a specification	constructing a model

The expression stays the same. What changes is where the candidates come from, what is done with those that fit, and *who* answers: a selecting or a generating backend.

Two questions to keep in mind

- **Does the robot know everything?** The query treats what it knows as complete; last week's OWL reasoner did not. Who is right about `unclosed_1`? (Act 1)
- **What happens when the world changes?** A cook makes a mistake at 19:50. Which of the robot's conclusions still hold? (Act 4)

How we work today

- In pairs, one laptop. The **sheet** is for logic and predictions, the **notebook** for running EQL.
- **Predict, then run.** Write your answer on the sheet before you run a cell: a wrong prediction is where you learn the most.
- Every cell runs as it is; a `TODO` you cannot solve does not stop you.
- Every answer is drawn on the picture of the pizzeria. Draw along on Figure 1 of the sheet.

Before the acts: these slides, 8 minutes.

Prologue 4′	**Ask** 22′	**Describe** 10′	**Constrain** 14′	**Infer** 17′	**Generate** 10′	**Wrap** 5′